



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Ingeniería Del Software II

Sección 1

Liderazgo, Equipos y Cultura de Ingeniería

Profesor:

Márquez Félix

Grupo: Ingenio y Código

Aguilera Rhixeidys V-30.851.503

Bravo Yonkeiner V-30.994.057

Carreño Brayan V-32.015.527

Garcia Nicole V-30.809.865

Ciudad Guayana, septiembre de 2026

Introducción

La ingeniería de software moderna ha evolucionado hacia una disciplina socio-técnica donde la estructura organizacional y la comunicación dictan la arquitectura del sistema. Este informe aborda el rediseño estratégico de equipos de desarrollo y la gestión del talento humano, fundamentado en la Ley de Conway, el marco de *Team Topologies* y los principios de agilidad.

El documento se divide en dos secciones principales. En la primera, se analiza cómo los silos funcionales en una entidad bancaria colapsan la agilidad y fuerzan un diseño monolítico. Como solución, se aplica la "Maniobra Inversa de Conway" para reestructurar a la organización en equipos orientados al flujo de valor (*Stream-aligned*) que faciliten la transición hacia microservicios.

En la segunda sección, se evalúa el impacto crítico de la Seguridad Psicológica en el rendimiento técnico. A través de principios de Liderazgo de Servicio y Comunicación No Violenta, se establecen estrategias para mitigar conflictos destructivos, objetivar el *feedback* en las revisiones de código (Pull Requests) y restaurar la confianza del equipo. En conjunto, este trabajo demuestra que alcanzar la excelencia en el desarrollo de software requiere, indispensablemente, diseñar un entorno cultural e interactivo sano.

Pregunta 1.: La Maniobra Inversa de Conway y Team Topologies Contexto

1.1 Análisis del problema bajo la Ley de Conway

La estructura organizacional actual del banco está garantizando el fracaso de la agilidad debido a que opera mediante silos funcionales ("Departamento de Base de Datos", "Departamento de Backend Java" y "Departamento de UI"), lo que genera graves ineficiencias operativas.

- **Aplicación de la Ley de Conway:**

Este principio establece que las organizaciones están limitadas a diseñar sistemas que son copias exactas de sus estructuras de comunicación. Al tener equipos aislados por capa tecnológica, el banco está forzado orgánicamente a producir y mantener una arquitectura acoplada y monolítica.

- **Fricción y pérdida de agilidad:**

Esta separación por componentes genera cuellos de botella y dependencias constantes. Cada vez que se intenta lanzar la funcionalidad de "Créditos", se producen transferencias constantes de contexto (handoffs) entre los departamentos, altos tiempos de espera en colas de trabajo y una visión fragmentada del producto.

- **Dilución de la responsabilidad:**

Al no existir un equipo con autonomía de extremo a extremo, la responsabilidad sobre el resultado final en producción se diluye, requiriendo la intervención burocrática de un gerente para coordinar la entrega.

1.2 Rediseño organizacional mediante la Maniobra Inversa de Conway y Team Topologies

Para soportar una futura migración a microservicios, el banco debe aplicar la **Maniobra Inversa de Conway** (Inverse Conway Maneuver). Esto implica estructurar proactivamente los equipos alrededor de los contextos delimitados del negocio (como "Créditos") para que las barreras de comunicación coincidan con las futuras interfaces de los microservicios (APIs).

Utilizando el marco de Team Topologies, los equipos quedarían conformados de la siguiente manera:

- **Equipos Alineados al Flujo de Valor (Stream-aligned teams):**

Se debe crear un equipo multifuncional dedicado exclusivamente al dominio de "Créditos". Este equipo integrará a especialistas con perfiles en forma de "T" provenientes de los antiguos departamentos funcionales (UI, Backend Java y Bases de Datos). Tendrán autonomía de extremo a extremo (end-to-end) para diseñar, construir y operar la funcionalidad de créditos sin requerir aprobaciones cruzadas constantes, bajo el principio de "you build it, you run it".

- **Equipos de Plataforma (Platform teams):**

Para evitar que el equipo de "Créditos" sufra de sobrecarga cognitiva manejando la infraestructura subyacente, se debe crear un equipo de plataforma. Este equipo proveerá servicios auto-atendidos (como despliegues automatizados, pipelines de CI/CD y aprovisionamiento de bases de datos como servicio) abstrauyendo la complejidad infraestructural.

- **Equipos de Subsistemas Complicados (Complicated-subsystem teams):**

Se formarán para manejar dominios del banco que requieran altísima especialización técnica o matemática. En este contexto, podrían encargarse del motor central de cálculo de riesgo crediticio o de los algoritmos de cifrado y seguridad financiera, permitiendo que el equipo de "Créditos" simplemente consuma estos servicios.

- **Equipos Facilitadores (Enabling teams):**

Actuarán como consultores internos temporales para cerrar brechas de conocimiento. Ayudarán al antiguo personal de backend monolítico a incorporar nuevas tecnologías y adoptar prácticas modernas como el Diseño Guiado por el Dominio (DDD) o la arquitectura de microservicios.

1.3 El nuevo rol del Gerente de Proyecto

En esta nueva estructura orientada al flujo de valor, el rol tradicional del Gerente de Proyecto (basado en el comando, control y la asignación de tareas a través de silos) queda obsoleto. Su función mutará hacia la de un **Líder Servicial y Adaptativo** (como un Scrum Master o Agile Coach).

- **Liderazgo de Servicio:**

El líder dejará de coordinar transferencias de código entre departamentos funcionales para enfocarse en eliminar impedimentos, proteger al equipo de interferencias externas y proveer el contexto necesario para que el equipo de "Créditos" se autoorganice.

- **Liderazgo Adaptativo:**

Ayudará al equipo a enfrentar los "desafíos adaptativos", tales como el cambio cultural necesario para abandonar la mentalidad de silos y adoptar verdaderamente las prácticas de microservicios y DevOps. Su objetivo principal será cultivar la Seguridad Psicológica del equipo para fomentar la innovación.

Pregunta 2.: Liderazgo, Conflicto y Seguridad Psicológica Contexto

2.1 Diagnostico y Abordaje del Conflicto

2.1.1 Diagnóstico de la Situación utilizando la Seguridad Psicológica (Amy Edmondson)

Marco Teórico: La Dra. Amy Edmondson (Harvard Business School, 1999) define la Seguridad Psicológica como la creencia compartida por los miembros de un equipo de que el entorno es seguro para asumir riesgos interpersonales. En un equipo psicológicamente seguro, los miembros confían en que no serán rechazados, ridiculizados o sancionados por hacer preguntas, admitir que desconocen un tema, reportar fallos tempranos o proponer ideas arquitectónicas divergentes.

Evidencia de la Industria (Proyecto Aristotle de Google): La investigación empírica de Google a lo largo de cinco años, conocida como Proyecto Aristotle (2015), demostró tras analizar 180 equipos de ingeniería que la seguridad psicológica es el predictor estadístico número uno (#1) indiscutible del alto rendimiento y la excelencia técnica en software, superando ampliamente variables como el coeficiente intelectual individual, la experiencia acumulada o los recursos de infraestructura.

Diagnóstico del Desplazamiento Cultural: Cruzando los Estándares de Rendimiento con la Seguridad Psicológica (Modelo de Edmondson), el equipo posee estándares técnicos sumamente altos (exigidos por la brillantez del Arquitecto Principal), pero una seguridad psicológica severamente erosionada por sus comentarios sarcásticos y descalificadores. Esto desplaza al equipo directamente a la 'Zona de Ansiedad', mutando con rapidez hacia la 'Zona de Apatía'. Cuando un junior es ridiculizado en una herramienta pública y permanente como GitHub, el cerebro activa mecanismos de amenaza social y supervivencia. Por ende, el enmudecimiento de los juniors en las retrospectivas no es apatía laboral, sino una respuesta racional de autopreservación: el silencio defensivo para evitar ser humillados.

2.1.2 Impacto en el Rendimiento del Equipo a Largo Plazo

Análisis de Consecuencias: El impacto operacional y arquitectónico de no intervenir esta situación de raíz genera cuatro fallas estructurales:

- 1. Ocultamiento Sistemático de Bugs y Deuda Técnica:** Al imperar el miedo al escarnio, los desarrolladores noveles ocultan fallos tempranos, dudas conceptuales o malas decisiones de diseño. Prefieren parchar código a escondidas antes que consultar al arquitecto, acumulando deuda técnica silenciosa que detonará en fallos catastróficos en producción (efecto Chernobyl de software).
- 2. Colapso de Métricas DORA (Accelerate):** El Lead Time for Changes se infla drásticamente debido a la postergación defensiva en el envío de Pull Requests (los juniors tardan días por pánico a la revisión). La tasa de fallos de cambio (Change Failure Rate) se incrementa por falta de discusión abierta y colaborativa sobre los casos borde.
- 3. Parálisis por Análisis y Muerte de la Innovación:** El equipo se niega a innovar o refactorizar código monolítico obsoleto. Prefieren copiar y pegar patrones antiguos que el arquitecto ya haya tolerado, matando la innovación técnica.
- 4. Deserción y Pérdida Crítica de Talento:** Los ingenieros junior y mid-level renuncian ante la hostilidad interpersonal, generando altos costos de reconstrucción y pérdida crítica de conocimiento de dominio de negocio.

2.1.3 Estrategia de Intervención con Liderazgo de Servicio y Comunicación No Violenta (CNV)

Marco de Mediación: Como nuevo Agile Coach / Líder Técnico, la respuesta no debe ser punitiva ni una confrontación de egos, ya que el Arquitecto es una pieza técnica valiosa. Siguiendo el Liderazgo de Servicio (Greenleaf, 1977) y el Liderazgo Adaptativo (Heifetz, 2009), se aborda en una reunión privada 1 a 1 mediante los cuatro pasos de la Comunicación No Violenta (CNV) de Marshall Rosenberg:

- **Paso 1: Hechos Objetivos (Observación sin juicio):** «Aprecio profundamente tu excelencia técnica y tu rol central en la arquitectura de nuestros sistemas. Revisando los Pull Requests de esta semana, observé este comentario puntual en el PR #48 del módulo de pagos: "¿En serio vas a usar este patrón? Es de principiantes"». (Se describe el hecho específico sin adjetivos personales como tóxico o agresivo).
- **Paso 2: Sentimientos (Impacto genuino):** «Al leer ese comentario y notar que en las dos últimas retrospectivas los juniors guardaron silencio total y la velocidad de entrega cayó un 30%, me siento sumamente preocupado por la sostenibilidad del equipo y la moral colectiva».
- **Paso 3: Necesidades Compartidas (Valores comunes):** «Comparto plenamente tu necesidad de mantener un código robusto y sin fallos en producción; sin embargo, para que el sistema escale sin que tú te conviertas en un cuello de botella que deba revisar línea por línea, necesitamos que el equipo aprenda sin miedo y gane autonomía. La excelencia técnica necesita un ambiente seguro para florecer».
- **Paso 4: Petición Concreta y Negociable:** «Quiero pedirte dos compromisos puntuales: primero, que en los PRs sustituyamos los juicios de valor por preguntas pedagógicas y alternativas de código (explicando por qué un patrón no es óptimo en este caso); y segundo, que me ayudes a co-diseñar un nuevo estándar de revisión de código con linters automáticos y Conventional Comments para que tú te enfoques en mentoría de alto nivel y no en corregir sintaxis. ¿Cómo lo ves y cómo podemos trabajar juntos en esto?».

2.2 Reestructuración Pull Requests y Retrospectiva

2.2.1 Reestructuración del Proceso de Pull Requests para Objetivar el Feedback

Estrategia Sistémica: Para que el cambio no dependa únicamente de voluntades individuales, se rediseña el proceso técnico de revisión en tres niveles:

- 1. Despersonalización Mediante Automatización en CI (Quality Gates):** Las discusiones sobre formato, sangrías, estilos y malas prácticas elementales se delegan a herramientas automáticas antes de que un humano intervenga: configurando Prettier y ESLint en hooks pre-commit y SonarQube en el pipeline de CI. Axioma rector: 'Las máquinas revisan el estilo y errores comunes; los humanos debaten la arquitectura, lógica de negocio y legibilidad'. Si el commit viola una regla sintáctica, es rechazado algorítmicamente sin fricción interpersonal.
- 2. Adopción de Conventional Comments:** Se implementa como política obligatoria el estándar internacional de Conventional Comments (conventionalcomments.org), exigiendo prefijos semánticos que explicitan la intención del revisor:

Etiqueta Semántica	Propósito / Intención	Ejemplo de Aplicación Constructiva
praise:	Elogio sincero por una buena solución técnica.	praise: Excelente desacoplamiento en este adaptador de pagos. Es muy limpio.
suggestion (no-bloq):	Propuesta de mejora técnica con código de ejemplo.	suggestion (no-bloqueante): Considera usar el patrón Factory aquí para evitar el switch múltiple. Te dejo este snippet: [...]
issue (bloqueante):	Problema funcional, de seguridad o rendimiento que bloquea el merge.	issue (bloqueante): Este bucle ejecuta consultas SQL dentro de una iteración (N+1). Puede causar saturación en la BD.

question:	Pregunta abierta para entender la intención del autor.	question: ¿Por qué elegiste un bloqueo síncrono en lugar de un canal asíncrono para este procesamiento? Me gustaría entender el caso de uso.
nitpick (pulido):	Detalle menor de formato o nombres no bloqueante.	nitpick: Podríamos renombrar esta variable a 'esValido' para mayor claridad semántica.

3. Código de Conducta en Revisiones (Code Review Charter): Se formaliza un acuerdo de equipo: 1) Criticar el código, jamás a la persona (prohibido sarcasmo o descalificaciones); 2) Toda observación crítica debe ir acompañada obligatoriamente de una solución o referencia técnica; 3) Mantener Micro-Pull Requests de menos de 400 líneas revisados en menos de 4 horas.

2.2.2 Dinámica Práctica para la Próxima Retrospectiva: Reconstruyendo la Confianza

Diseño de la Dinámica: Se diseña una sesión de Retrospectiva estructurada de 60 minutos en tres fases consecutivas para sanar el clima del equipo:

- **Fase 1: Safety Check Anónimo (10 minutos):** Inspirado en el marco de Derby & Larsen (2006), cada integrante vota anónimamente del 1 al 5 su grado de seguridad psicológica para hablar (1: terror al escarnio, 5: total libertad y confianza). Si el promedio es inferior a 3.5, el facilitador suspende la revisión de historias técnicas para atender exclusivamente la convivencia y el apoyo interpersonal.
- **Fase 2: Reconocimiento Cruzado y Normalización del Error (35 minutos):** Se aplica la dinámica 'Hero, Highlight & Hardship' combinada con Kudo Cards. Cada miembro debe entregar una tarjeta de agradecimiento pública a otro compañero por su apoyo en el sprint. El Arquitecto Principal tiene un rol protagónico reconociendo el

esfuerzo y los aciertos técnicos de los desarrolladores juniors, validando su valor profesional. Asimismo, el líder técnico modela vulnerabilidad compartiendo un error propio del pasado para desmitificar la infalibilidad técnica.

- **Fase 3: Co-creación del Team Working Agreement (15 minutos):** El equipo redacta y firma colectivamente sus Acuerdos de Convivencia y Mentoría (Team Working Agreement), el cual se versiona en el repositorio para normar las revisiones de código, la comunicación empática y la resolución constructiva de conflictos.

Conclusión

A partir del análisis desarrollado, se concluye que la eficiencia y escalabilidad en la ingeniería de software no dependen exclusivamente de la adopción de nuevas tecnologías, sino de una profunda alineación entre la estructura organizacional, la arquitectura del sistema y la cultura del equipo. La Ley de Conway demuestra empíricamente que los modelos tradicionales basados en departamentos funcionales estancos son incompatibles con la agilidad moderna; generan fricción, cuellos de botella y fuerzan el acoplamiento técnico. Para superar este paradigma, la Maniobra Inversa de Conway, apoyada por las estructuras de Team Topologies (Stream-aligned, Platform, Enabling, Complicated-subsystem), se posiciona como el estándar estratégico indispensable para transformar jerarquías rígidas en ecosistemas orientados al flujo continuo de valor y arquitecturas descentralizadas como los microservicios.

Paralelamente, se evidenció que la excelencia técnica carece de sostenibilidad sin una sólida base de Seguridad Psicológica. Como demostró el caso analizado y las investigaciones empíricas de la industria, entornos de alta hostilidad o miedo al escarnio como los propiciados por comentarios destructivos en revisiones de código paralizan la innovación, inflan la deuda técnica y ahuyentan el talento. La intervención efectiva exige líderes adaptativos y serviciales capaces de objetivar el feedback mediante herramientas sistémicas (como la automatización de CI y Conventional Comments) y de fomentar espacios seguros a través de la Comunicación No Violenta. En síntesis, liderar proyectos de software en la actualidad es, fundamentalmente, diseñar un entorno organizacional y psicológico óptimo que libere el potencial creativo y técnico de los equipos, garantizando así la entrega de valor sin fricciones.

Referencias Bibliográficas

- Edmondson, A. C. (1999). Psychological Safety and Learning Behavior in Work Teams. *Administrative Science Quarterly*, 44(2), 350-383.
- Edmondson, A. C. (2018). *The Fearless Organization: Creating Psychological Safety in the Workplace for Learning, Innovation, and Growth*. John Wiley & Sons.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- Greenleaf, R. K. (1977). *Servant Leadership: A Journey into the Nature of Legitimate Power and Greatness*. Paulist Press.
- Heifetz, R., Grashow, A., & Linsky, M. (2009). *The Practice of Adaptive Leadership*. Harvard Business Press.
- Rosenberg, M. B. (2015). *Nonviolent Communication: A Language of Life* (3rd ed.). PuddleDancer Press.
- Derby, E., & Larsen, D. (2006). *Agile Retrospectives: Making Good Teams Great*. Pragmatic Bookshelf.
- Duhigg, C. (2016). What Google Learned From Its Quest to Build the Perfect Team (Project Aristotle). *The New York Times Magazine*.